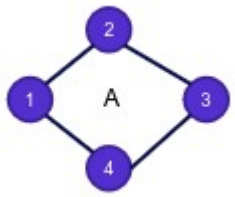
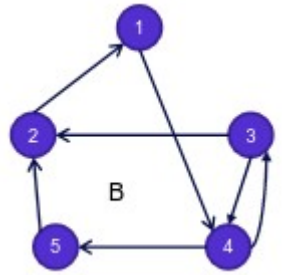
Tipos de Grafos

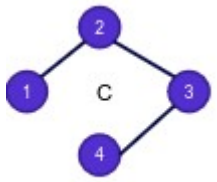
● Grafo **Trivial**: Formado por um único vértice
● Grafo **Simples**: Não direcionado, sem laços e sem arestas paralelas.
● Grafo **Completo**: Grafo simples onde cada vértice se conecta a todos os outros vértices.
● Grafo **Cíclico**: Contém ciclo: caminho que começa e termina sempre no mesmo vértice.
● Grafo **Regular**: todos os vértices possuem o mesmo grau.
● Grafo com **Múltiplas** Arestas: Permite mais de uma aresta conectando o mesmo par de vértices (Arestas Paralelas)
● Grafo **Acíclico**: Não contém ciclo
● Grafo **Bipartido**: Dois conjuntos de vértices. Vértices do mesmo conjunto não são ligados



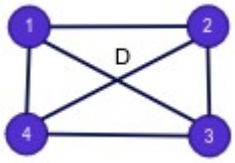
Não direcionado;
Simples;
Regular;
Cíclico;
Bipartido;



Direcionado;
'Cíclico;
Bipartido;

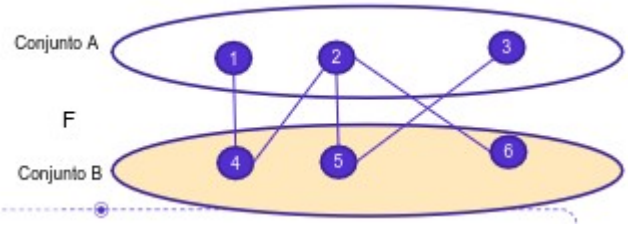


Não direcionado;
Simples;
Acíclico;

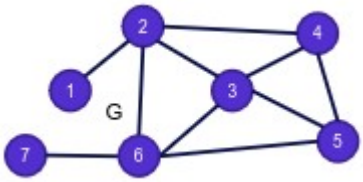


Não direcionado;
Simples;
Completo;
Regular;
Cíclico;

Trivial;
Regular;



Não direcionado;
Simples;
Cíclico;
Bipartido;



Não direcionado;
Simples;
Cíclico;

Outros tipos

Isomorfo: existe uma função que faça o mapeamento de vértices e arestas de modo que o dois grafos se tornem coincidentes.

Ponderado: possui peso associado a cada uma das arestas

Hamiltoniano: possui um caminho que visita cada vértice apenas uma vez

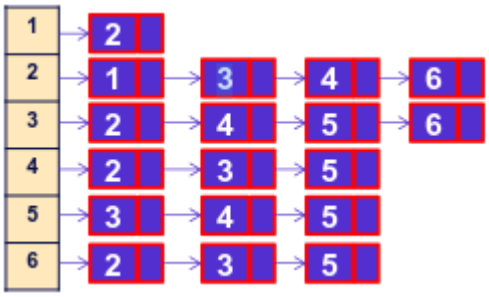
Grafo Eureliano: Possui ciclo que visita cada aresta apenas uma vez;
Pode passar por um vértice mais de uma vez, buscando passar por todas as arestas.
O grafo deve ser direcionado
RESUMO: Tem que passar por todos os nós; tem que passar por todos os caminhos; não pode repetir caminhos; pode repetir nós;

Representações

Matriz de Adjacência (muitas ligações)

(i,j)	1	2	3	4	5	6
1	0	1	0	0	0	0
2	1	0	1	1	0	1
3	0	1	0	1	1	1
4	0	1	1	0	1	0
5	0	0	1	1	0	1
6	0	1	1	0	1	0

Lista de Adjacência (poucas ligações)



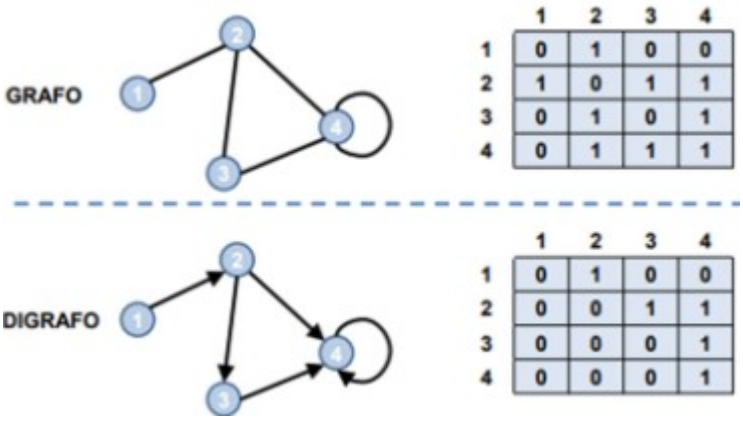
Representação de Grafo – Matriz de Adjacência

● Utiliza uma matriz N x N para armazenar o grafo, onde N é o número de vértices

○ Alto custo computacional, $O(N^2)$

● Uma aresta é representada por uma marca na posição (i , j) da matriz

○ Aresta liga o vértice i ao j



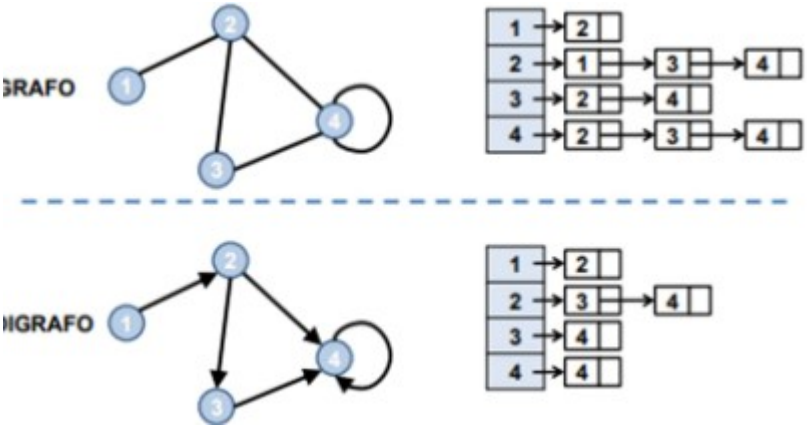
	1	2	3	4
1	0	1	0	0
2	1	0	1	1
3	0	1	0	1
4	0	1	1	1

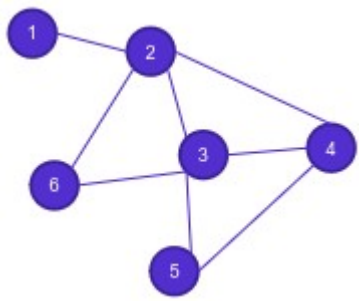
	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	0	0	0	1

Representação de Grafo – Lista de Adjacência

● Utiliza uma lista de vértices para descrever as relações entre os vértices.

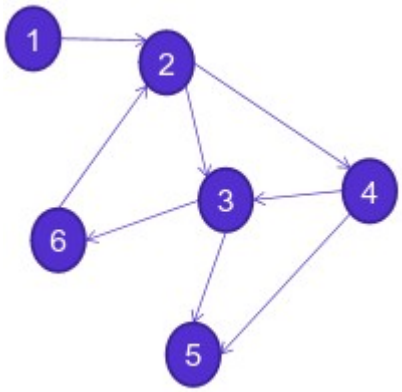
○ Um grafo contendo N vértices utiliza um array de ponteiros de tamanho N para armazenar os vértices do grafo
○ Para cada vértice é criada uma lista de arestas, onde cada posição da lista armazena o índice do vértice a qual aquele vértice se conecta





	1	**2**	**3**	**4**	**5**	**6**
1	0	1	0	0	0	0
2	1	0	1	1	0	1
3	0	1	0	1	1	1
4	0	1	1	0	1	0
5	0	0	1	1	0	0
6	0	1	1	0	0	0

| 1 | → | 2 |
| 2 | → | 1 | → | 3 | → | 4 | → | 6 |
| 3 | → | 2 | → | 4 | → | 5 | → | 6 |
| 4 | → | 2 | → | 3 | → | 5 |
| 5 | → | 3 | → | 4 |
| 6 | → | 2 | → | 3 |

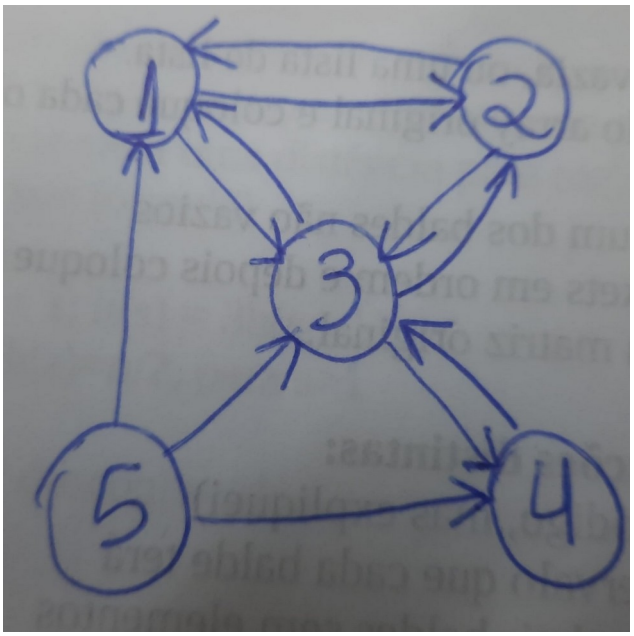


	1	**2**	**3**	**4**	**5**	**6**
1	0	1	0	0	0	0
2	0	0	1	1	0	0
3	0	0	0	0	1	1

4	0	0	1	0	1	0
5	0	0	0	0	0	0
6	0	1	0	0	0	0

| 1 | → | 2 |
| 2 | → | 3 | → | 4 |
| 3 | → | 5 | → | 6 |
| 4 | → | 3 | → | 5 |
| 5 | → nullptr
| 6 | → | 2 |

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{vmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{vmatrix} \end{array}$$



Aula 08/04

Busca em Profundidade – BFS

A patir de um nó, busca saber um caminho “mais longo”.
Descobrir o maior caminho.

1	→	2	→	5
2	→	3
3	→	1	→	4
4	→	5
5	→	3

Declara vetor e cont

Aqui, o nó inicial foi escolhido o número 1.

```
BFS (G, 1)                                  cont = 1
    1.2 BFS (G, 2)                          cont = 2
        2.3 BFS (G, 3)                      cont = 3
            3.1 BFS (G, 1)                  cont = 4
            3.4 BFS (G, 4)                  cont = 4
                4.5 BFS (G, 5)              cont = 5
                    5.3 BFS (G, 3)          cont = 6
                    FIM 5
                FIM 4
            FIM 3
        FIM 2
    1.5 BFS (G, 5)
    FIM 1
```

descubro → no visitado pela primeira vez.

Cont = 0

0	0	0	0	0
0	1	2	3	4

Cont = 1; no = 1

1	0	0	0	0
0	1	2	3	4

Cont = 2; no = 2; cont = 2

1	2	0	0	0
0	1	2	3	4

Cont = 2; no = 3; cont = 3

1	2	3	0	0
0	1	2	3	4

Cont = 3; no = 1; cont = 4

1	2	3	4	0
0	1	2	3	4

Cont = 4; no = 5; cont = 5

1	2	3	4	5
0	1	2	3	4

Aqui, o nó inicial foi escolhido o número 3.

1	→	2	→	5
2	→	3
3	→	1	→	4
4	→	5
5	→	3

Cont = 0

0	0	0	0	0
1	2	3	4	5

no = 3; cont = 1

0	0	1	0	0
1	2	3	4	5

no = 1; cont = 2

2	0	1	0	0
1	2	3	4	5

no = 2; cont = 3

2	3	1	0	0

1	2	3	4	5

no = 3; cont = 4

2	3	1	0	4
1	2	3	4	5

no = ?; cont = 5

2	3	1	5	4
1	2	3	4	5

Busca em Largura – BFS

Irá criar um vetor e uma fila;
Na fila, o primeiro que entra, é o primeiro que sai.

1	→	2
2	→	3	→	5
3	→	1	→	4
4	→	1
5	→

0	0	0	0	0
1	2	3	4	5

Fila: 1 ← **Aqui, o nó inicial foi escolhido o número 1.**
Analizar e inserir os vizinhos de 1.

Fila: 1x , 2
Fila: 2x , 3 , 5
Fila: 3x , 5 , 1 , 4
Fila: 5x , 1 , 4
Como 1 já foi analizado, pula para o próximo
Seria: Fila: 1x , 4 Mas o 1 já foi analizado
Fila: 4x

Fila: 1

1	0	0	0	0
1	2	3	4	5

Fila: 1x , 2

1	2	0	0	0
1	2	3	4	5

Fila: 2x , 3 , 5

1	2	3	0	0
1	2	3	4	5

Fila: 3x , 5 , 4

1	2	3	0	3

1	2	3	4	5

Fila: 5x , 4

1	2	3	4	3
1	2	3	4	5

Fila: 4x

1	2	3	4	3
1	2	3	4	5

Exemplo para aplicar a profunidade e a largura

1	→	2
2	→	3	→	5
3	→	1	→	4
4	→	1
5	→

Aqui, o nó inicial foi escolhido o número 1.

PROFUNDIDADE

BFS (G, 1)	cont = 1
1.2 BFS (G, 2)	cont = 2
2.3 BFS (G, 3)	cont = 3
3.1 BFS (G, 1)	→ 1 já foi chamado
3.4 BFS (G, 4)	cont = 4
4.1 BFS (G, 1)	→ 1 já foi chamado
FIM 4	
FIM 3	
2.5 BFS (G, 5)	cont = 3
FIM 5	→ 5 não aponta para ninguém
FIM 2	
FIM 1	

Aqui, o nó inicial foi escolhido o número 1.

cont = 0

0	0	0	0	0
1	2	3	4	5

No: 1; v[1] = (cont++); cont = 1; v[1] = 1

1	0	0	0	0
1	2	3	4	5

Filho de nó 1 ← No: 2; v[2] = (cont++); cont = 2; v[2] = 2

1	2	0	0	0
1	2	3	4	5

Filho de nó 2 ← No: 3; v[3] = (cont++); cont = 3; v[3] = 3

1	2	3	0	0
1	2	3	4	5

→ Nó 3 aponta para 1; mas 1 já foi visitado;

Filho de nó 3 ← No: 4; v[4] = (cont++); cont = 4; v[4] = 4

1	2	3	4	0
1	2	3	4	5

→ Nó 4 aponta para 1; mas 1 já foi visitado;

→ Terminou a recursão do 4; volta para o 3;
→ Terminou a recursão do 3; volta para o 2;

→ Na recursão do nó 2, ainda resta o 5 (ainda não foi analizado).

Filho de nó 2 ← No: 5; v[5] = (cont++); cont = 3; v[5] = 3

1	2	3	4	3
1	2	3	4	5

LARGURA

| 1 | → | 2 |
| 2 | → | 3 | → | 5 |
| 3 | → | 1 | → | 4 |
| 4 | → | 1 |
| 5 | →

Quantas camadas para chegar (a partir de um determinado nó) até outro (verifica todos) nó.

Irá criar um vetor e uma fila;
Na fila, o primeiro que entra, é o primeiro que sai.

Fila: 1 ← **Aqui, o nó inicial foi escolhido o número 1.**
Analizar e inserir os vizinhos de 1.

Fila: 1

1	0	0	0	0
1	2	3	4	5

→ Add o 2

Fila: 1x , 2 cont = 1 Remove o 1 e add seus vizinhos que ainda n foram

1	2	0	0	0
1	2	3	4	5

→ Add o 3 e 5

Fila: 1x, 2x , 3 , 5 cont = 2 Remove o 2 e add seus vizinhos que ainda n foram

1	2	3	0	3
1	2	3	4	5

Fila: 1x, 2x , 3x , 5 cont = 3 Remove o 3 e add seus vizinhos que ainda n foram

1	2	3	0	3
1	2	3	4	5

→ Add o 4

Fila: 1x, 2x , 3x , 5x , 4 Remove o 5 e add seus vizinhos que ainda n foram

1	2	3	4	3
1	2	3	4	5

→ Add ninguem

Fila: 1x, 2x , 3x , 5x , 4x Remove o 4 e add seus vizinhos que ainda n foram

1	2	3	4	3
1	2	3	4	5

→ Add ninguem